

# N★SQL

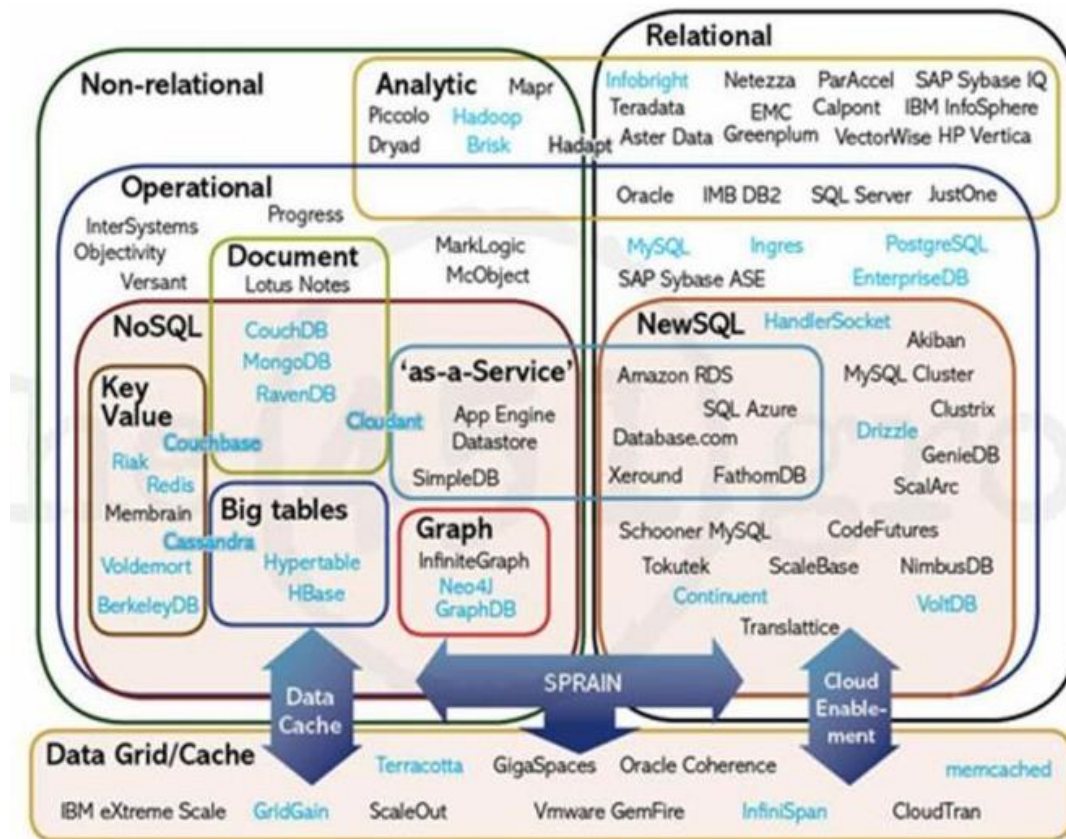
NOSQL

Redis  
Cassandra  
Voldemort  
Key/Value  
EC2  
MapReduce  
memcached  
MongoDB  
Azure  
Hadoop  
Riak  
CouchDB  
SimpleDB  
Cloud  
Neo4J  
BerkeleyDB  
CAP  
HBase

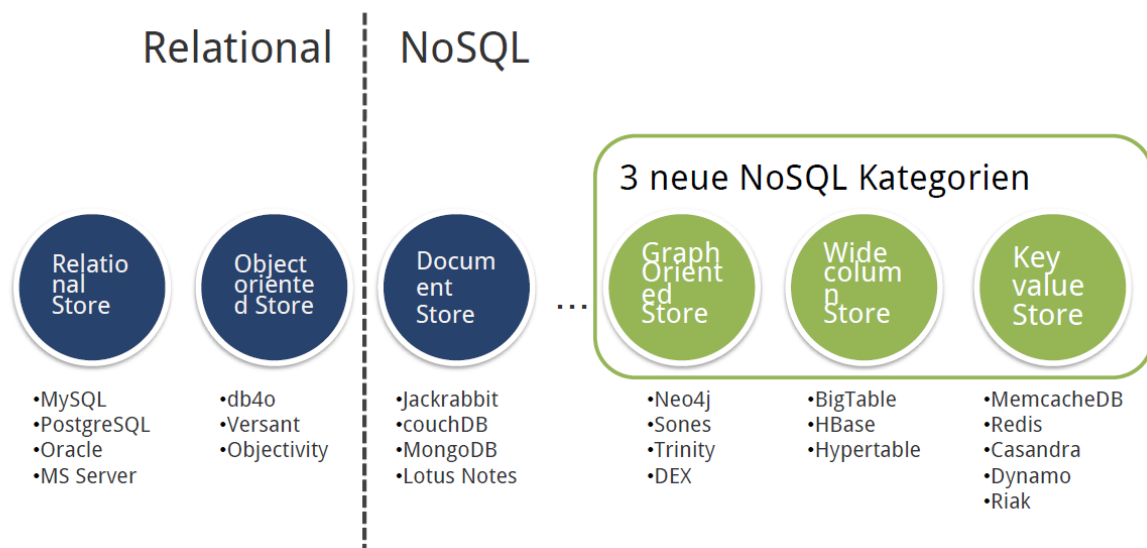
## NoSQL

“Not Only SQL”

# 1. DATENBANK-MARKENÜBERSICHT



## 1.1. KATEGORIEN



## 1.2. WAS IST NOSQL

- kein Mainstream
- Kein Produkt, keine Firma, keine Technologie
- Ist eine Bewegung, eine neu Art Datenbanken voranzutreiben
- Basiert auf nicht-relationale Ansätzen

## 2. WARUM NOSQL

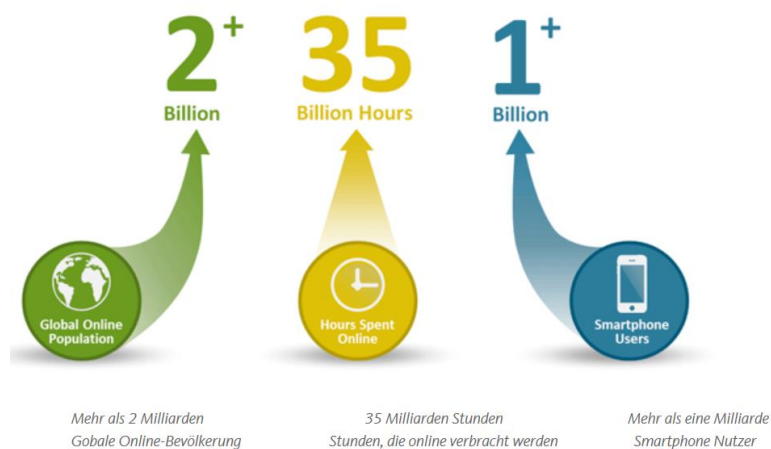
Interaktive Anwendungen haben sich in den letzten 15 Jahren rasant verändert und ebenso das Datenmanagement dieser Anwendungen. Momentan sind es drei zusammenhängende Megatrends, die für eine Verbreitung der NoSQL-Technologie sorgen: Big Data, Big User und Cloud Computing. NoSQL wird zunehmend als echte Alternative zu relationalen Datenbanken gesehen, da immer mehr Unternehmen realisieren, dass Daten auf Clustern von Standardserverumgebungen viel besser skalierbar sind und ein schemaloses Datenmodell oft besser für die Art und Menge an Daten ist, die heute erfasst und verarbeitet werden.

Drei Trends sind der Grund für eine Veränderung des Status Quo bei Datenbanken:

- Big Users
- Big Data
- Cloud Computing

### 1.3. BIG USERS

Es ist noch nicht allzu lange her, da waren 1.000 User für eine Anwendung sehr viel und 10.000 User bedeuteten einen Extremfall. Heute ist es nicht ungewöhnlich, dass Apps Millionen von Usern am Tag haben. Das liegt vor allem daran, dass das Internet global immer intensiver genutzt wird, die User viel mehr Stunden online verbringen und dass Smartphones und Tablets immer beliebter werden.



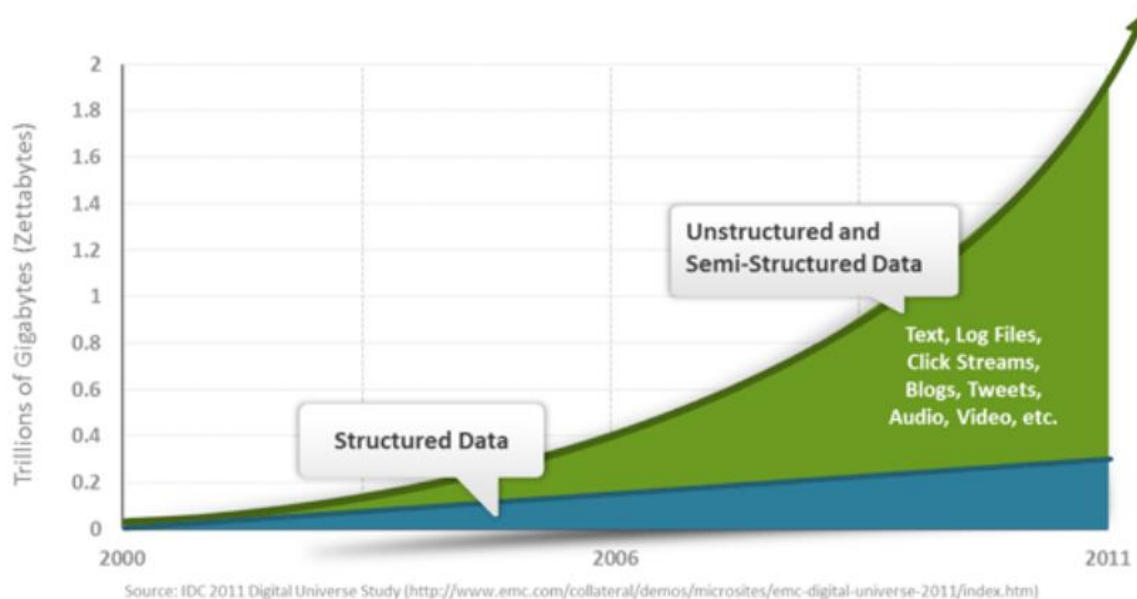
Es ist wichtig, dass eine grosse Anzahl an gleichzeitigen Usern unterstützt wird. Gerade aber weil die Anforderungen an die Nutzung der Apps so unvorhersehbar sind, ist es genauso wichtig, eine schnell wachsende (oder sinkende) Zahl an gleichzeitigen Usern dynamisch unterstützen zu können.

- Eine neu gestartete App kann viral werden und die Anzahl der User über Nacht buchstäblich von Null auf eine Million ansteigen.
- Manche User sind häufig aktiv, andere nutzen die App nur ein paar Mal und dann nie wieder.
- Saisonale Schwankungen, beispielsweise um Weihnachten oder den Valentinstag, können für kurze Zeit zu einer Spitzenauslastung führen.

Genau diese hohe Anzahl an Usern und das dynamische Nutzerverhalten sind der Grund dafür, dass der Bedarf an leichter skalierbaren Datenbanktechnologien steigt. Für viele Anwendungsentwickler ist es schwierig oder beinahe unmöglich, mit relationalen Technologien die nötige dynamische Skalierbarkeit und Größe zu erreichen und dabei gleichzeitig die benötigte konstant verfügbare Leistung aufrecht zu erhalten, so wie es das Nutzervolumen erfordert. In diesem Moment ist NoSQL die Rettung.

## 1.4. BIG DATA

Durch Drittanbieter wie Facebook, D&B und andere wird es immer einfacher, Zugang zu Daten zu erhalten und diese zu verarbeiten. Persönliche Benutzerinformationen, Geo-Positionsdaten, soziale Graphen, nutzergenerierte Inhalte, Maschinendatenerfassung, und von Sensoren generierte Daten sind nur ein paar Beispiele für die ständig wachsende Menge an Daten, die erfasst wird. Es überrascht daher nicht, dass Entwickler aus diesem Grund bestehende Anwendungen verbessern und neue erstellen möchten. Die Nutzung dieser Daten verändert vieles von Grund auf und in einem hohen Tempo: Kommunikation, Einkaufen, Werbung, Unterhaltung und die Kundenbeziehung. Apps, die diesem Trend nicht Rechnung tragen, werden schnell ins Hintertreffen geraten.



*Billionen von Gigabytes (Zettabytes)*  
*Strukturierte Daten*  
*Unstrukturierte und semistrukturierte Daten*  
*Text, Logdateien, Click Streams, Blogs, Twitter-Nachrichten, Audio, Video, etc.*

Entwickler benötigen sehr flexible Datenbanken, die schnell neue Arten von Daten aufnehmen und die nicht durch Struktur und Content-Änderungen von Drittdatenanbietern beeinträchtigt werden können. Da eine Menge dieser neuen Daten unstrukturiert und semistrukturiert ist, brauchen Entwickler außerdem Datenbanken, die diese Daten effizient speichern können. Die durch ein Datenbankschema fest vorgegebene Struktur von relationalen Datenbanken ist leider ungeeignet, um neue Arten von Daten schnell hinzuzufügen und stellt daher eine denkbar schlechte Lösung für unstrukturierte und semistrukturierte Daten dar. NoSQL bietet ein Datenmodell, das diesen Anforderungen viel besser gerecht wird.

## 1.5. CLOUD COMPUTING

Heutzutage basieren die meisten neuen Anwendungen (sowohl für Privatanwender als auch Unternehmen) auf einer 3-Schichten-Internetarchitektur. Sie laufen auf einer öffentlichen oder privaten Cloud und unterstützen eine große Anzahl an Usern.



In dieser Drei-Schichten-Architektur gelangt man über einen Webbrowser oder eine mobile App, die mit dem Internet verbunden ist, auf die Anwendungen. In der Cloud leitet ein Load-Balancer den eingehenden Datenverkehr zu einer vertikal skalierten Schicht von Web/Anwendungsservern, die die Logik der Anwendung verarbeiten. Auf der Web-Anwendungsschicht funktioniert die Scale-Out Architektur perfekt. Pro 10.000 (oder eine beliebige andere Anzahl) neue gleichzeitig aktive User wird einfach ein weiterer Standardserver zu der Webanwendung hinzugefügt, um die Last aufzufangen.

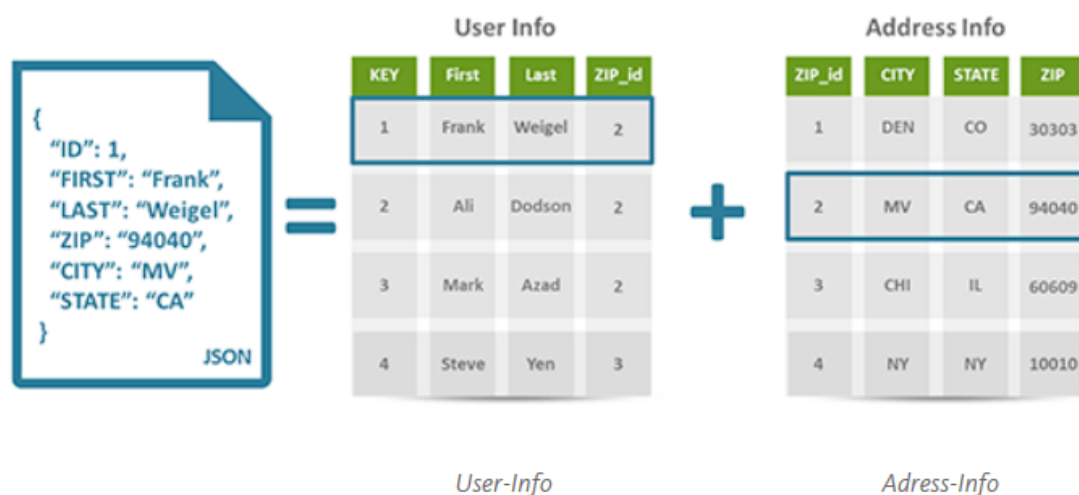
Auf der Datenbankebene waren relationale Datenbanken zunächst eine beliebte Wahl. Ihr Einsatz wurde jedoch zunehmend problematischer, da sie auf einer zentralisierten, Share-Everything Technologie basieren, die eher Möglichkeiten zum Vertikalen Skalieren als zum Horizontalen Skalieren bieten. Daher waren sie keine optimale Lösung für Anwendungen, die einfach und dynamisch skalierbar sein müssen. NoSQL-Datenbanken wurden von Grund auf als verteilte, horizontal skalierbare Datenbanken entwickelt und passen daher viel besser auf die hochgradig verteilte 3-Schichten-Internetarchitektur.

### 3. WARUM ENTSCHEIDEN SICH ENTWICKLER FÜR NOSQL?

#### 1.6. EIN FLEXIBLERES DATENMODELL

Relationale und NoSQL-Datenmodelle unterscheiden sich erheblich. Im relationalen Modell werden Daten erfasst und in viele miteinander zusammenhängende Tabellen unterteilt, die Zeilen und Spalten enthalten. Die Tabellen enthalten Verweise und sind durch Fremdschlüssel, die ebenfalls in Spalten gespeichert werden, miteinander verbunden. Wenn Daten abgerufen werden, muss die gewünschte Information aus vielen Tabellen (bei heutigen Unternehmensanwendungen sind dies oft Hunderte) zusammengestellt und verknüpft werden, bevor sie für die Anwendung bereitgestellt werden kann. Ebenso muss beim Schreiben von Daten darauf geachtet werden, dass diese in vielen Tabellen erfasst und ausgeführt werden.

NoSQL-Datenbanken verfolgen ein komplett anderes Modell. Eine dokumentenorientierte NoSQL-Datenbank übernimmt beispielsweise die Daten, die gespeichert werden sollen und aggregiert sie in Dokumente im JSON-Format. Jedes JSON-Dokument kann als ein Objekt betrachtet werden, das von der Anwendung genutzt wird. In einem JSON-Dokument können beispielsweise alle Daten aus einer Zeile aus 20 Tabellen einer relationalen Datenbank vereint und in einem einzigen Dokument/Objekt aggregiert werden. Die Aggregation dieser Informationen kann dazu führen, dass Duplikate entstehen. Dies wird bei webbasierten Anwendungen jedoch gerne in Kauf genommen, da Datenspeicher erschwinglich geworden sind, das Datenmodell dadurch sehr flexibel ist, die Dokumente effizienter und einfacher verteilt werden können und der Lese- und Schreibzugriff performanter wird.



Ein weiterer wichtiger Unterschied ist, dass relationale Technologien mit einem festen Schema funktionieren, während NoSQL-Modelle schemalos sind. Relationale Technologien benötigen eine klare Definition eines Schemas, bevor überhaupt Daten in einer Datenbank gespeichert werden können. Sobald einmal Daten eingegeben sind, sind Änderungen am Schema sehr schwierig und beeinträchtigen die Funktionsweise enorm, daher geht man diesen Änderungen gern aus dem Weg. Dies steht jedoch genau im Gegensatz zu dem, was in der Big Data-Ära eigentlich notwendig ist. Anwendungsentwickler müssen ständig und in hoher Geschwindigkeit neue Arten von Daten hinzufügen, um ihre Anwendungen zu verbessern.

Im Vergleich dazu sind Dokumentendatenbanken schemalos, wodurch bei JSON-Dokumenten einfach Felder hinzugefügt werden können, ohne dass vorher Änderungen definiert werden müssen. Das einzugebende Datenformat kann jederzeit verändert werden, ohne dass die Anwendung dabei beeinträchtigt wird.

### Vergleich relationale vs. Document orientiertes Datenmodell

|      |      |      |      |
|------|------|------|------|
|      |      |      |      |
| R1C1 | R1C2 | R1C3 | R1C4 |
| R2C1 | R2C2 | R2C3 | R2C4 |
| R3C1 | R3C2 | R3C3 | R3C4 |
| R4C1 | R4C2 | R4C3 | R4C4 |
|      |      |      |      |

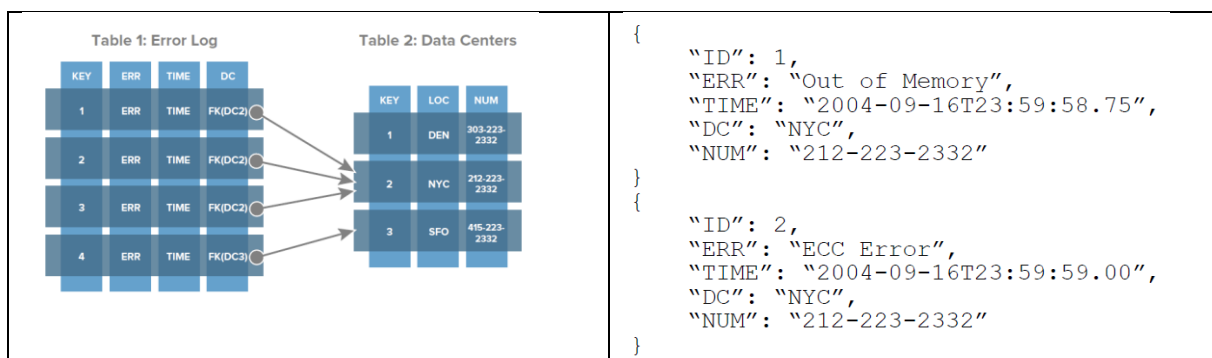
#### Relational data model

Highly-structured table organization with rigidly-defined data formats and record structure.



#### Document data model

Collection of complex documents with arbitrary, nested data formats and varying "record" format.

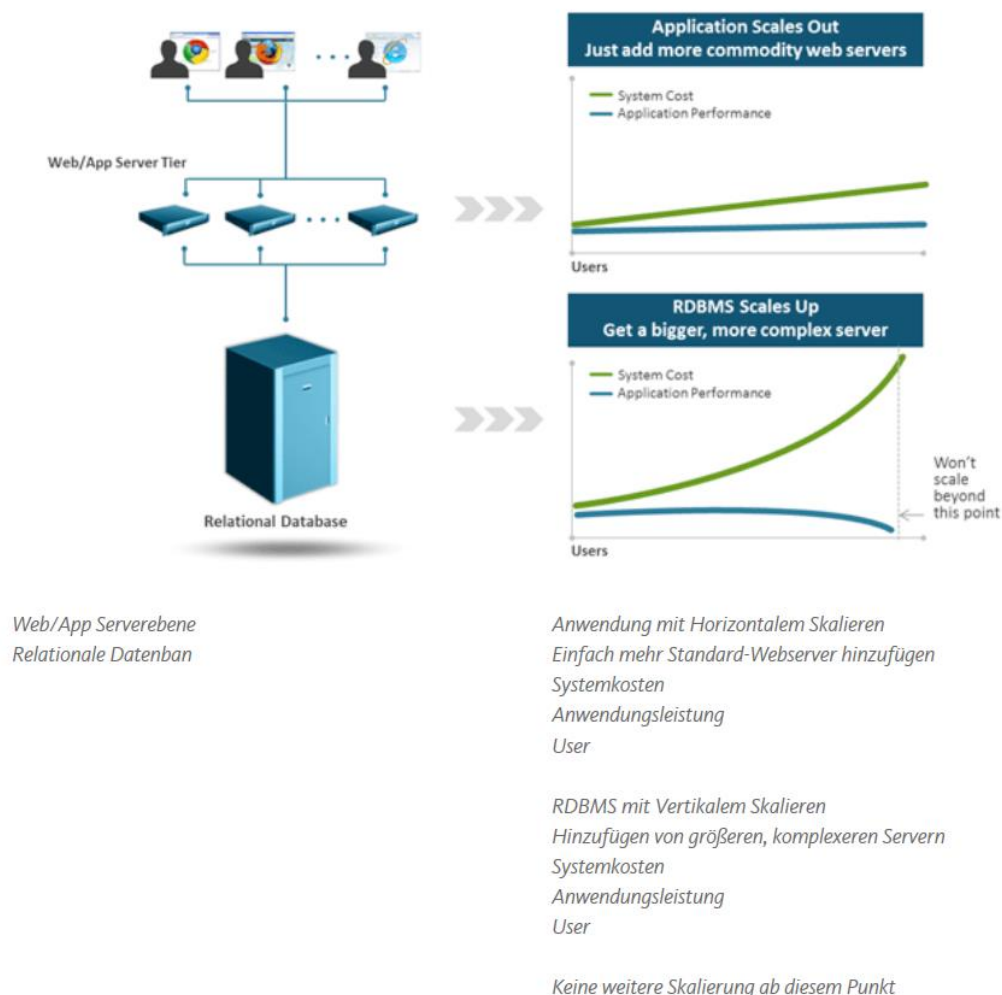


## 1.7. SKALIERBARKEIT UND LEISTUNGSSTEIGERUNG

Um die steigende Zahl an gleichzeitigen Usern (Big Usern) und die Menge an Daten (Big Data) bewältigen zu können, gibt es **zwei Möglichkeiten der Skalierung**. Anwendungen und ihre zugrundeliegenden Datenbanken können **vertikal** oder **horizontal** skaliert werden (Scale-Up oder Scale-Out). Beim vertikalen Skalieren wird **zentralisiert aufgerüstet** und in immer grössere und leistungsfähigere Server investiert. **Beim horizontalen Skalieren wird eine verteilte Strategie verfolgt**, bei dem vielen Standardserver, physische oder virtuelle Server verbunden werden.

### VERTIKALES SKALIEREN MIT RELATIONALER TECHNOLOGIE: GRENZEN AUF DATENBANKEBENE

Auf der Web-Anwendungsebene der 3-Schichten-Internetarchitektur wurde die Strategie des Horizontalen Skalierens über viele Jahre hinweg standardmässig eingesetzt und dies funktionierte auch sehr gut. Je mehr User eine Anwendung nutzen, desto mehr Standardserver werden auf der Web-Anwendungsebene hinzugefügt. Die Leistung bleibt dabei konstant, da die Last auf einer höheren Anzahl an Servern verteilt wird.



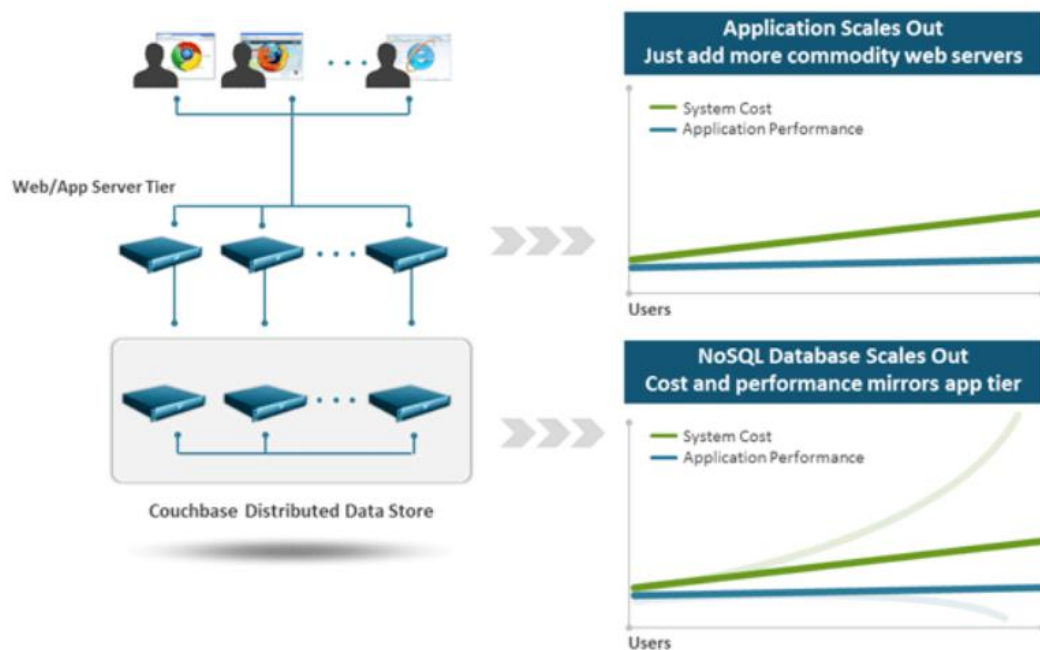
Bevor es NoSQL-Datenbanken gab, war das vertikale Skalieren der am meisten verbreitete Ansatz auf Datenbankebene. Dies war durch die durchweg zentralisierte, Shared-Everything Architektur der relationalen Datenbanktechnologie vorgegeben. Um eine größere Anzahl an gleichzeitigen Usern zu unterstützen und mehr Daten zu speichern, werden immer grössere und leistungstärkere Server benötigt. Mehr CPUs sowie größere Arbeits- und

Festplattenspeicher sind notwendig, um alle Tabellen erhalten zu können. Große Server sind in der Regel sehr komplex, proprietär und unverhältnismäßig teuer im Gegensatz zu der kostengünstigen Standard-Hardware, die meist so effektiv auf der Web -Anwendungsserver-Ebene eingesetzt wird.

### HORIZONTALES SKALIEREN MIT NOSQL-TECHNOLOGIE AUF DATENBANKEBENE

**NoSQL-Datenbanken wurden von Grund auf als verteilte, auf Scale-Out-Technologie basierende Datenbanken entwickelt.** Mehrere Standards, physische oder virtuelle Server werden als Cluster zusammengeschlossen, um Daten zu speichern und Datenbankvorgänge zu unterstützen. Für eine Skalierung werden dem Cluster zusätzliche Server hinzugefügt und die Daten sowie Datenbankvorgänge verteilen sich somit auf dem größeren Cluster. Da man damit rechnen muss, dass Standardserver von Zeit zu Zeit ausfallen, wurden NoSQL-Datenbanken robust entwickelt, so dass sie einen derartigen Ausfall aushalten und damit sehr ausfallsicher sind.

NoSQL-Datenbanken bieten eine viel einfachere, lineare Lösung zur Datenbank-Skalierung. Wenn auf einmal 10.000 neue User Ihre Anwendung nutzen, kann einfach ein weiterer Datenbankserver an das Cluster angeschlossen werden. Kommen weitere 10.000 User, fügen Sie den nächsten Server hinzu. Die Anwendung muss bei einer Skalierung nicht verändert werden, da die Anwendung immer nur eine einzige (verteilte) Datenbank wahrnimmt.



Web/App Serverebene

Couchbase Verteilter Datenspeicher

Anwendung mit Horizontalem Skalieren

Einfach mehr Standard-Webserver hinzufügen

Systemkosten

Anwendungsleistung

User

NoSQL-Datenbank mit Horizontalem Skalieren Kosten und

Leistung spiegeln Anwendungsebene wider

Systemkosten

Anwendungsleistung

User

- **Auto-Sharding** - Eine NoSQL-Datenbank verteilt die Daten automatisch auf verschiedene Server, ohne dass dabei Anwendungen einbezogen werden. Server können der Datenebene hinzugefügt oder von ihr entfernt werden, ohne dass es zu Ausfallzeiten der Anwendung kommt. Die Daten (und I/O) werden automatisch auf die Server verteilt. Die meisten NoSQL-Datenbanken unterstützen auch eine Replikation der Daten. Mehrere Kopien der Daten können innerhalb des Clusters, oder sogar über mehrere Rechenzentren hinweg, gespeichert werden. Somit ist eine hohe Verfügbarkeit und Unterstützung für eine Disaster Recovery gewährleistet. Bei einem korrekt verwalteten NoSQL-Datenbanksystem ist es aus keinem Grund und zu keinem Zeitpunkt nötig, das System offline zu stellen, da der Betrieb aller Anwendungen konstant und Rund-um-die-Uhr gewährleistet ist.
- Unterstützung verteilter Abfragen - „**Sharding**“ kann bei relationalen Datenbanken in bestimmten Fällen dazu führen, dass komplexe Datenabfragen nicht mehr oder nur noch in geringem Maße durchgeführt werden können. In NoSQL-Datenbanksysteme bleiben die vollständigen Abfragemechanismen erhalten, selbst wenn sie auf Hunderten von Servern verteilt sind.
- Integriertes Caching - Um die Latenz niedrig zu halten und einen hohen Datendurchsatz zu gewährleisten, bieten hoch entwickelte NoSQL-Datenbanktechnologien ein transparentes Caching von Daten im Systemspeicher. Die ermöglicht eine vollständige Transparenz für die Anwendungsentwickler und das Operations-Team, während bei relationalen Datenbanktechnologien die Caching-Ebene meist über eine eigene Infrastrukturebene verfügt, die auf separaten Servern entwickelt, eingesetzt und explizit von den Ops-Teams verwaltet wird.

## 4. ARTEN VON NOSQL-DATENBANKEN / KLASSIFIZIERUNG

### KEY-VALUE-STORES

- Key-Value-Lookups
- Optimiert auf konkurrierende Zugriffe
- DynamoDB, Riak, Voldemort



### 1.8. COLUMN FAMILIES

- Jeder Schlüssel assoziiert mit mehreren Attributen (Columns)
- Schlüssel / Wert-Paare bilden Spalten
- Unterschiedliche Anzahl Spalten / Zeile
- Keine Joins
- Kein Schema
- Sehr grosse verteilte Datenmengen
- BigTable, Cassandra



### 1.9. DOCUMENT DATABASE (DOKUMENTENORIENTIERTE DATENBANKEN)

- Semistrukturierter Dokumente
- JSON oder BSON
- Kein Schema / Keine Joins
- Komplexe Datenstrukturen
- CouchDB, MongoDB

Dokumente...

```
{ "name": "Meier",  
  "forename": "Max",  
  "adress": { "street": "Deich 7",  
             "postcode": 28355,  
             "city": "Bremen" },  
  "comment": "Good old boy." }
```



### 1.10. GRAPH DBs (GRAPHENORIENTIERTE DATENBANKEN)

- Verbindungen zwischen Objekten
- Knoten
- Gerichtete Beziehungen
- Attribute an Knoten und Beziehungen
- Rekursive Strukturen
- Kein Schema
- Neo4j



## 5. UNTERSCHIED: QUERIES

- Keine Relationen
  - keine Konsistenz untereinander
- Keine AdHoc-Queries
  - außer Keys/vorberechnete Daten
- Keine Konsistenz
  - Eventual Consistency

Ein Schritt zurück?

- Scalability, availability and resilience come at a cost
- Erfordert andere Programmiertechniken



### 1.11. GEMEINSAMKEITEN IN DER NOSQL-WELT

- Kein Schema
- Kein ACID-Support
- ...aber BASE!
- Hohe Skalierbarkeit

### 1.12. NoSQL VOR- UND NACHTEILE

Vorteile:

- Hohe Skalierbarkeit
- Hohe Verfügbarkeit
- Geringe Kosten
- Vorhersagbare Elastizität
- Flexible Schemata
- Alle NoSQL-Datenbanken outperformen RDBMS

Nachteile:

- Beschränkte Query-Möglichkeiten
- Eventual Consistency
- Kein Standard (a la SQL)

## 6. FROM ACID TO CAP

### 1.13. EIGENSCHAFTEN VON RDBMS :

- **Atomicity** – ganz oder gar nicht
- **Consistency** – gewährleistet Integrität
- **Isolation** – Kapselung gleichzeitiger T.
- **Durability** – Persistenz aller Änderungen

Skalieren vertikal-> grosse Datenbanken ->teuer

### 1.14. DAS CAP-THEOREM

